TransitionSetバッチ処理パネル UI仕様書

概要

目的

組織内全MAIMLファイルのセマンティック検索インデックス（4軸ベクトルEmbedding）の一括生成を管理するパネルコンポーネント。バッチジョブの開始・監視・キャンセル機能を提供する。

使用場面

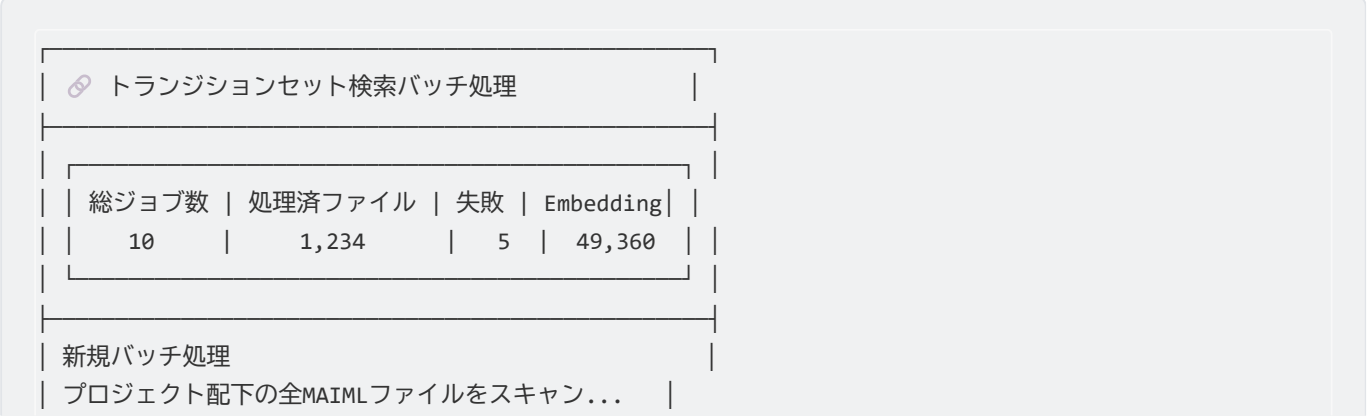
- TransitionSetSearchModalの「バッチ処理」タブ内
- MAIMLファイルの一括インデックス生成
- バッチジョブの進捗監視とエラー確認

アクセス権限

- **Experiment Admin（実験管理者）**: バッチ処理の実行・監視
- **その他のロール**: アクセス不可

画面構成

レイアウト構造



```
│                                                  │
│ 既存ファイルを強制再処理: [OFF]                  │
│ 最大処理ファイル数（省略可）: [全ファイル]       │
│                                                  │
│ [▶ バッチ処理を開始]                             │
├──────────────────────────────────────────────────┤
│ バッチジョブ一覧                          [🔄]   │
│ ┌──────────────────────────────────────────────┐ │
│ │ Job #a1b2c3d4...  [処理中] [キャンセル]    │ │
│ │ 進捗: 50 / 100 ファイル                      │ │
│ │ [━━━━━━━━━━]  50%                            │ │
│ │ [処理済: 50] [失敗: 2] [スキップ: 10]       │ │
│ │ 作成: 2026/03/11 10:00  完了: -              │ │
│ └──────────────────────────────────────────────┘ │
│ ┌──────────────────────────────────────────────┐ │
│ │ Job #e5f6g7h8...  [完了]                     │ │
│ │ [処理済: 100] [スキップ: 0]                  │ │
│ │ 作成: 2026/03/11 09:00  完了: 09:15          │ │
│ └──────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────┘
```

UI要素

1. ヘッダー

- **アイコン**: 🔗 `NodeIndexOutlined`（24px、青色）
- **タイトル**: 「トランジションセット検索バッチ処理」（level 5）

2. 統計カード

- **背景色**: `#f0f5ff`（淡い青）
- **サイズ**: small
- **表示項目**:
  - 総ジョブ数: `stats.total_jobs`
  - 処理済ファイル: `stats.total_files_processed`
  - 失敗ファイル: `stats.total_files_failed`（0より大きい場合は赤色）
  - 生成Embedding数: `stats.total_embeddings_generated`

3. 新規バッチ処理カード

- **タイトル**: 「新規バッチ処理」
- **説明**: 「プロジェクト配下の全MAIMLファイルをスキャンし、4軸ベクトルEmbeddingを生成します。」（12px、グレー）
- **設定項目**:
  - 強制再処理スイッチ: ON/OFF
  - 最大処理ファイル数: InputNumber（1以上、省略可）
- **開始ボタン**: primaryタイプ、large、`PlayCircleOutlined`アイコン、全幅

4. バッチジョブ一覧カード

- **タイトル**: 「バッチジョブ一覧」
- **Extra**: 更新ボタン（`ReloadOutlined`アイコン、ローディング時は回転）
- **リストアイテム**: 各ジョブの詳細情報

機能仕様

State管理

```
// 処理状態
const [isStarting, setIsStarting] = useState(false);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
const [successMessage, setSuccessMessage] = useState<string | null>(null);

// バッチ設定
const [forceReprocess, setForceReprocess] = useState(false);
const [maxFiles, setMaxFiles] = useState<number | null>(null);

// データ
const [jobs, setJobs] = useState<BatchJob[]>([]);
const [stats, setStats] = useState<BatchStats | null>(null);
```

主要機能

1. データ取得（ジョブ一覧と統計）

```
const fetchData = useCallback(async () => {
  try {
    setIsLoading(true);
    const [jobList, batchStats] = await Promise.all([
      listBatchJobs(),
      getBatchStats(),
    ]);
    setJobs(jobList);
    setStats(batchStats);
  } catch (err) {
    console.error('[BatchPanel] データ取得エラー:', err);
  } finally {
    setIsLoading(false);
  }
}, []);
```

実行タイミング:

- コンポーネントマウント時
- 更新ボタンクリック時
- バッチ開始・キャンセル後

2. 自動ポーリング

```
useEffect(() => {
  const hasActive = jobs.some(
    (j) => j.status === 'started' || j.status === 'processing'
  );
  if (!hasActive) return;
  const timer = setInterval(fetchData, 3000);
  return () => clearInterval(timer);
}, [jobs, fetchData]);
```

動作:

- アクティブなジョブ（started/processing）が存在する場合
- 3秒ごとに自動更新
- アクティブなジョブがなくなると停止

3. バッチ開始

```
const handleStart = async () => {
  setError(null);
  setSuccessMessage(null);
  setIsStarting(true);
  try {
    const options: BatchIngestOptions = {
      forceReprocess,
      maxFiles: maxFiles ?? null,
    };
    const result = await startBatch(options);
    setSuccessMessage(`バッチ処理を開始しました（Job ID: ${result.batch_job_id}）`);
    await fetchData();
  } catch (err) {
    setError(err instanceof Error ? err.message : 'バッチ処理の開始に失敗しました');
  } finally {
    setIsStarting(false);
  }
};
```

動作:

1. エラー・成功メッセージをクリア
2. `isStarting` を `true` に設定
3. バッチオプションを構築
4. `startBatch()` でバッチ開始
5. 成功メッセージを表示
6. データを再取得
7. エラー時はエラーメッセージを表示

4. ジョブキャンセル

```
const handleCancel = async (jobId: string) => {
  try {
    await cancelBatchJob(jobId);
    setSuccessMessage(`ジョブ ${jobId} をキャンセルしました`);
    await fetchData();
  } catch (err) {
    setError(err instanceof Error ? err.message : 'キャンセルに失敗しました');
  }
};
```

動作:

1. `cancelBatchJob()` でジョブをキャンセル
2. 成功メッセージを表示
3. データを再取得
4. エラー時はエラーメッセージを表示

5. 進捗率の計算

```
const calcProgress = (job: BatchJob): number => {
  if (job.total_files === 0) return 0;
  return Math.round((job.processed_files / job.total_files) * 100);
};
```

計算式: `(processed_files / total_files) × 100` （整数）

データ型定義

BatchIngestOptions

```
interface BatchIngestOptions {
  forceReprocess: boolean;              // 既存ファイルを強制再処理
```

```
  maxFiles: number | null;            // 最大処理ファイル数（null: 全ファイル）
}
```

BatchJob

```
interface BatchJob {
  id: string;                         // ジョブID
  status: 'started' | 'processing' | 'completed' | 'failed' | 'cancelled'; // ステータス
  total_files: number;                // 総ファイル数
  processed_files: number;            // 処理済ファイル数
  failed_files: number;               // 失敗ファイル数
  skipped_files: number;              // スキップファイル数
  error_message?: string;             // エラーメッセージ
  created_at: string;                 // 作成日時
  completed_at?: string;              // 完了日時
}
```

BatchStats

```
interface BatchStats {
  total_jobs: number;                 // 総ジョブ数
  total_files_processed: number;      // 総処理済ファイル数
  total_files_failed: number;         // 総失敗ファイル数
  total_embeddings_generated: number; // 総生成Embedding数
}
```

ステータス表示

ステータスタグの色

```
function statusColor(status: BatchJob['status']): string {
  switch (status) {
    case 'started':
    case 'processing':
```

```
      return 'blue';
    case 'completed':
      return 'green';
    case 'failed':
      return 'red';
    case 'cancelled':
      return 'default';
    default:
      return 'default';
  }
}
```

ステータス	色	意味
started	blue（青）	開始中
processing	blue（青）	処理中
completed	green（緑）	完了
failed	red（赤）	エラー
cancelled	default（グレー）	キャンセル済

ステータスラベル

```
function statusLabel(status: BatchJob['status']): string {
  switch (status) {
    case 'started':
      return '開始中';
    case 'processing':
      return '処理中';
    case 'completed':
      return '完了';
    case 'failed':
      return 'エラー';
    case 'cancelled':
      return 'キャンセル済';
    default:
      return status;
  }
}
```

ジョブリストアイテム

表示内容

ヘッダー行:

- ジョブID（先頭8文字 + "..."）
- ステータスタグ
- キャンセルボタン（処理中のみ）

進捗表示（処理中のみ）:

- テキスト: 「進捗: {processed_files} / {total_files} ファイル」
- プログレスバー: パーセント表示、青色

統計タグ:

- 処理済: {processed_files}
- 失敗: {failed_files}（0より大きい場合のみ、赤色）
- スキップ: {skipped_files}（0より大きい場合のみ、オレンジ色）

エラーメッセージ（存在する場合）:

- Alertコンポーネント（error、banner）

日時情報:

- 作成: {created_at}（日本語ロケール）
- 完了: {completed_at}（完了済みの場合のみ）

API連携

1. バッチ開始

サービス: `startBatch(options: BatchIngestOptions)`

リクエスト:

```
{
  forceReprocess: boolean;
  maxFiles: number | null;
}
```

レスポンス:

```
{
  batch_job_id: string;
}
```

2. ジョブ一覧取得

サービス: `listBatchJobs()`

レスポンス: `BatchJob[]`

3. 統計取得

サービス: `getBatchStats()`

レスポンス: `BatchStats`

4. ジョブキャンセル

サービス: `cancelBatchJob(jobId: string)`

レスポンス: `void`

特記事項

自動ポーリング

条件:

- アクティブなジョブ（started または processing）が存在する

動作:

- 3秒ごとに `fetchData()` を実行
- アクティブなジョブがなくなると自動停止
- メモリリークを防ぐためのクリーンアップ実装

キャンセルボタンの表示

表示条件:

```
(job.status === 'started' || job.status === 'processing')
```

- 処理中のジョブのみキャンセルボタンを表示
- 完了・失敗・キャンセル済みのジョブは表示しない

空状態の表示

ローディング中:

- スピナーを中央に表示

ジョブ0件:

- 「バッチジョブがありません」を表示

実装ファイル情報

パス

```
frontend/src/components/TransitionSetSearch/TransitionSetBatchProcessingPanel.tsx
```

主要な依存パッケージ

- `antd` : UI コンポーネント（Button, Card, Typography, Space, Switch, InputNumber, Alert, List, Tag, Progress, Spin, Statistic, Row, Col, Divider, Tooltip）
- `@ant-design/icons` : アイコン（PlayCircleOutlined, StopOutlined, ReloadOutlined, NodeIndexOutlined）
- `react` : React本体（useState, useEffect, useCallback）

サービス

- `semanticSearchService` : バッチ処理API

変更履歴

バージョン	日付	変更内容	担当者
1.0	2026-03-11	実装ベースで仕様書を新規作成（実装ファイル: TransitionSetBatchProcessingPanel.tsx）	Cline

---

関連ドキュメント

- TransitionSet検索モーダル
- 検索結果一覧
- 重み調整スライダー
- プロジェクト管理画面